	YES	NO	N/A	Comments/Evidence/Rationale
Input Validation				
All entry points and trust boundaries are identified by the design.	X			Using dev tools, I was able to map the hackazon site.
Input validation is applied whenever input is received from outside the current trust boundary.		X		Search attempts to look for anything input in the field. Even if it is out of bounds.
The design assumes that user input is malicious.		X		SQL injections are possible in login fields.
Centralized input validation is used where appropriate.		X		There does not appear to be any centralized validation. Each form seems to handle it independently.
The input validation strategy that the application adopted is modular and consistent.		X		Validation is inconsistent. Some fields validate; others don't. The search field allows for values other fields don't.
The validation approach is to constrain, reject, and then sanitize input. Looking for known, valid, and safe input is much easier than looking for known malicious or dangerous input.			X	Local hosted Hackazon does not have a server to contact for sanitization.
Data is validated for type, length, format, and range.	X			Data is validated. I cannot use letters in number fields. Cannot go above 10 digits in a length capped field. ETC.
The design addresses potential canonicalization issues.		X		After messing around with the URL encoding the site it opened with multiple different addresses.

Input file names and file paths are avoided where possible.			X	Hackazon doesn't have a file upload feature.
The design addresses potential SQL injection issues.		X		The site is vulnerable to SQL injections. I was able to force a popup with XSS using SQL injection on the log in page.
The design addresses potential cross-site scripting issues.		X		The site is vulnerable to scrip issues. Using <script>alert(1)</script> in the item search gives a pop up that says 1.
The design does not rely on client-side validation.		X		It seems that the site relies on some level of client-side validation.
The design applies defense in depth to the input validation strategy by providing input validation across tiers.		X		No evidence of DiD. Invalid inputs can affect the site.
Output that contains input is encoded using HtmlEncode and UrltEncode.			X	Unable to test.
Configuration Management				
Administration interfaces are secured (strong authentication and authorization is used).		X		The admin login page is viewable even while on a user account.
Remote administration channels are secured.			X	Untestable as hackazon is running local on a VM. No outside internet connection.
Configuration stores are secured.			X	I am unable to access the configuration files on the VM. Thus, I cannot test this

Configuration secrets are not held in plain text in configuration files.			X	I am unable to access the configuration files on the VM. Thus, I cannot test this
Administrator privileges are separated based on roles (for example, site content developer or system administrator).			X	Hackazon doesn't have multiple administrator roles.
Least-privileged process accounts and service accounts are used.	X			When logged in under a user account the admin logs in page gave "You don't have permissions to access this resource."
Sensitive Data				
Secrets are not stored unless necessary. (Alternate methods have been explored at design time.)			X	Can not be tested on the VM. Hackazon does not have payment or password reset abilities.
Secrets are not stored in code.			X	Untestable, I don't have access to site code.
Database connections, passwords, keys, or other secrets are not stored in plain text.			X	No access to hackazon files.
The design identifies the methodology to store secrets securely. (Appropriate algorithms and key sizes are used for encryption. It is preferable that the Data Protection API (DPAPI) is used to store configuration data to avoid key management.)			X	I don't have access to the design documents. I am unable to say for certain.
Sensitive data is not logged in clear text by the application.	X			Sensitive data is not stored in clear text.

The design identifies protection mechanisms for sensitive data that is sent over the network.		X		Hackazon uses http, an unsecured communication.
Sensitive data is not stored in persistent cookies.	X			The cookies are stored as a hash.
Sensitive data is not transmitted with the GET protocol.		X		All data is transmitted via the GET protocol.
Parameter Manipulation				
All input parameters are validated (including form fields, query strings, cookies, and HTTP headers).		X		I was able to change search parameters. Length 0 to prevent input or changing the type to number.
Cookies with sensitive data are encrypted.	X			All the cookies are encrypted.
Sensitive data is not passed in query strings or form fields.	X			No sensitive data was transmitted during operation.
HTTP header information is not relied on to make security decisions.			X	It doesn't appear to be connected to the HTTP header. However, I am not certain.
View state is protected using MACs.			X	Out of scope for hackazon on the VM.

Security Re-Design Document

Application Name: Hackazon

Prepared by: William Arnold

Date: 2/20/2026

1. Summary

This document presents a security redesign of Hackazon. An assessment identified multiple security vulnerabilities in input validation, configuration validation and parameter manipulation. In phase 2 we defined security requirements including: authorization, credentials and access control, passwords, hashing, data integrity and protection. This redesign document translates those requirements into architectural changes and security implementations that align with best practices.

2. Identified Security Problems

The application had the following weaknesses:

- Input parameters were not validated.
- The site uses http, an unsecured communication over the internet.
- Admin login pages are available to the public.
- The site does not show signs of defense in depth.
- The design relies on client-side validation.

- The design does not address potential cross-site scripting issues.
- The design does not address potential SQL injection issues.
- The design does not address potential canonicalization issues.
- Input validation is not applied whenever input is received from outside the current trust boundary.
- The design does not assume that user input is malicious.
- Centralized input validation is not used.
- The input validation strategy is not modular and is inconsistent.

3. Security Requirements

- Access Rights
- User and Group Privileges
- Role-Based Access Control
- Credential Security
- Protection of Credentials
- Source Identification
- Integrity Checks
- Unique User ID

4. Design Changes

The following design changes address the identified security problems and reduce the attack surface of Hackazon by adding secure software design best practices. First, all communication between the user and server will use HTTPS with modern encryption. This will prevent data interception, session hijacking and information theft. Any unsecured access will be disabled to enforce the secure connection.

Second, all authentication controls will be strengthened. Password requirements will be updated to include a minimum number of characters and special characters. The admin log in page should be removed from the user side of the site. This will minimize its exposure to unauthorized users. Third, Hackazon should implement access rights. Currently, Hackazon has only two roles. User and administrator. More roles should be added, and they should follow the principle of least privilege. Authorization should be handled by the server to prevent client-side tampering.

Fourth, all input should be treated as malicious. Validation should enforce data types depending on the field, as well as length restrictions and acceptable formats. All inputs should be sterilized to prevent malicious inputs. Inputs should be normalized to prevent canonicalization attacks. Finally, validation logic should be modular and centrally managed. This ensures consistency and strengthens maintainability. With these changes, Hackazon will be much more secure. These changes follow the greater security requirements while still fixing the many issues that were found in the Hackazon site.

References:

Cybersecurity Best Practices for Software Development 2025: Secure by Design - Eucalipse | Eucalipse. (n.d.). https://eucalipse.com/articles/cybersecurity-software-development-best-practices-2025

GotQuestions.org. (2026, January 21). GotQuestions.org. https://www.gotquestions.org/biblical-stewardship.html

Group, J. T. F. I. W. (2020). Security and privacy controls for information systems and organizations. https://doi.org/10.6028/nist.sp.800-53r5

Input Validation - OWASP Cheat Sheet Series. (n.d.). https://cheatsheetseries.owasp.org/cheatsheets/Input_Validation_Cheat_Sheet.html